MODIFIED FLUIDNINJA CORE AND BUOYANCY PLUGIN GUIDE

FluidNinja architecture and buoyancy

How the modified core turns visible water into a surface that floating objects can feel, disturb, and share.

This guide explains the modified FluidNinja LIVE core and the reusable FluidNinja Buoyancy plugin. It connects the implementation to the physics, then walks through building a beach ball, a log, a boat, and a wrecking ball in your own Unreal project. You should finish knowing which settings control weight, floating depth, stability, current, waves, and visual effects.

CORE ENHANCEMENTS

Water reacts to objects

Relative tracking, propagating surface waves, independent foam, and optional sharper rendered crests.

BUOYANCY EXTENSION

Objects react to water

Collision-derived hulls, asynchronous surface sampling, Chaos forces, two-way wave coupling, presets, and diagnostics.

The central idea
The renderer draws water on the GPU. The plugin reads small patches of that water for each body, computes forces in Chaos, and sends the bodies' displacement and impacts back to the modified GPU core.

How to read this guide

Start with Architecture and The physics for the explanation. Use First floating object and the four object recipes to implement it. Keep the property reference open while tuning. The HTML includes a calculator and searchable properties; the PDF includes the same explanations and worked examples.

Source audit: 18 October 2026 · FluidNinja Buoyancy 1.0.1 source · Unreal Engine 5.8

Evidence boundary
This guide describes the inspected C++, HLSL, plugin configuration, and authoring scripts. It does not certify a particular project's runtime behavior. Check saved Blueprint values, the installed core version, and runtime performance in your project. The worked example results are calculations under stated assumptions.

READING AND REFERENCE

Contents

Read the architecture first, follow the object recipes, then use the property reference while implementing.

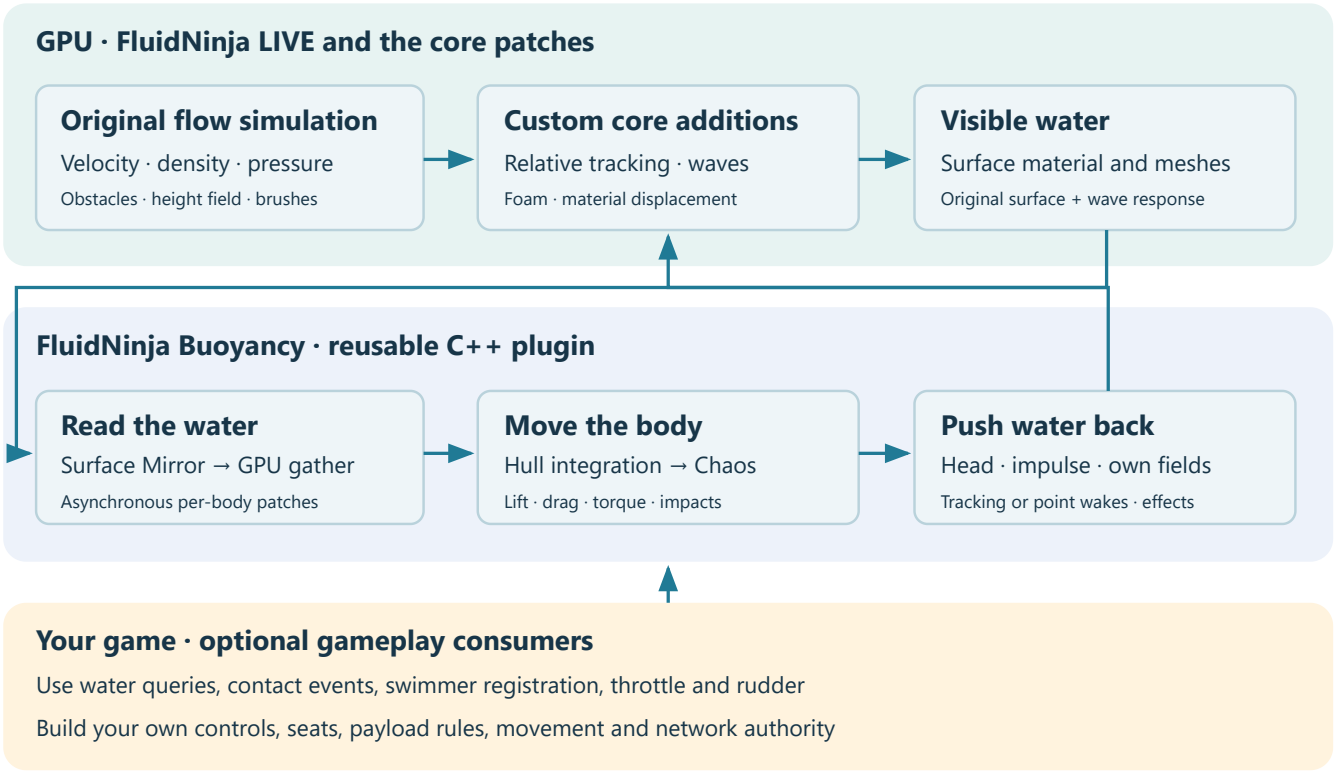
Topic	Page
The architecture	3
The FluidNinja core enhancements	4
From a rendered surface to a physics force	5
How objects make waves and feel other waves	7
Why a body does not ride its own delayed wave	8
The physics that makes an object float	9
Mass, draft, and stability are different controls	11
Create the first floating object	12
Choose values from size and mass	13
Beach ball and log recipes	14
A small boat and a wrecking ball	15
Buoyancy calculator	17
Add propulsion and steering to your boat	17
Configure the water that objects use	19
Waves, brush wakes, foam, and splashes	21
Queries, swimmers, and integration points	22
What the force integrator computes	24
Know when the setup is correct	25
Keep the plugin and core compatible	27
Property reference	28
Source map and glossary	44

Source defaults and calculated examples are labeled throughout. This guide does not claim a new runtime validation.

01 · OWNERSHIP AND TECHNOLOGY

The architecture

FluidNinja owns the GPU flow simulation and water rendering. The core patches extend its Niagara and material assets. The reusable C++ plugin connects that water to Chaos rigid bodies. Your game can consume the plugin's water queries, events, and propulsion controls for its own movement or vehicle systems.



Ownership diagram. The two upward/downward connections form a delayed water–body feedback loop. The flow simulation and the new wave field are separate systems.

Layer	Where it lives	Responsibility
FluidNinja LIVE	`/Game/FluidNinjaLive`	Niagara fluid grid, flow buffers, masks, inputs, surface rendering.
Custom core	Modified NinjaLiveCore, painter assets, Waves modules, SURFACE function	Relative tracking; wave, foam, and rendering extensions. Maintained HLSL/tooling lives in the buoyancy plugin.
Shared plugin	`Plugins/FluidNinjaBuoyancy`	Water readback, hull physics, self fields, queries, effects, presets, editor validation and developer tests.

Layer	Where it lives	Responsibility
Your game	Your own Blueprint or C++ classes	Optional consumers of the public APIs. Boarding, payload management, custom movement and network authority are gameplay responsibilities, not automatic buoyancy features.

Technology in plain language. Niagara runs parallel simulation stages on the GPU. HLSL supplies their math and compute shaders. Render targets hold grids of numbers. Unreal's render graph and asynchronous GPU readback move selected results to C++. Chaos integrates forces into rigid-body motion. Blueprints expose the authored objects and gameplay controls. [S4] [S7] [S1]

The runtime connects FluidNinja to Chaos directly. It does not require the FluidFlux solver or Unreal Water's buoyancy component.

02 · CORE CHANGES

The FluidNinja core enhancements

The vendor manual for LIVE 2.0 describes buoyancy as unsupported and identifies loss of point identity in its proposed GPU-to-CPU sampling path. The buoyancy plugin keeps per-body patches and identity under its own control. The core modifications add the feedback needed for objects to disturb water without uncontrolled current amplification. [Vendor manual]

Enhancement	Change	What users gain
Relative tracking	PainterSourceData and Ninja_Painter take tracked horizontal motion relative to the sampled water when enabled.	A floater moving with the current does not continually paint that same world velocity back into it.
Wave field	NinjaLiveCore gains a height/vertical-speed grid, source texture input, four stages, and versioned outputs.	Entry rings, displacement wakes, reflections, and waves that rock other bodies.
Surface material	The SURFACE function adds wave height to world-position offset and slopes to normals.	The new waves are visible on existing compatible FluidNinja surfaces.
Independent wave resolution	Interface 3 can run the wave grid at 1, 1/2, or 1/4 of pressure-grid width.	Quality and cost control without resizing the original pressure output.
Independent foam	Interface 4 carries a separate advected, decaying foam field.	Whitecaps, hull trails, and impact foam without injecting flow density to obtain whiteness.
Optional choppiness	Material-only horizontal crest sharpening, behind WaveChoppy.	Sharper-looking crests. It does not add lift or a new physical force.

Why relative motion matters

Carried by the river

Body 200 cm/s →

Water 200 cm/s →

Relative motion = 0

Pushed ahead by a motor

Body 300 cm/s →

Water 200 cm/s →

Relative motion = 100 cm/s →

Conceptual velocities. The actual core conversion also accounts for simulation substeps and coordinate mapping.

The patch adds `User.TrackVelocityRelativeToWater`. It subtracts the previous fluid velocity from the tracked horizontal motion, with a gate fading from 2 to 10 cm/s, and handles multiple Niagara ticks in one frame. It preserves vertical puncture motion and bypasses screen-position/camera-facing cases. This is a brush interaction model, not a resolved hull boundary in a 3D fluid solver. [S10]

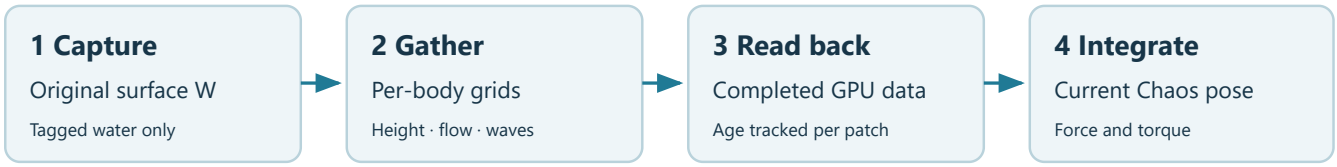
Velocity subtraction alone is insufficient. Tracking must also suppress tracked density using `SelectivelyKillBrushDensityKeepVelocity` = 1, 4, or 5. Otherwise density can strengthen the river driver. Value 5 also suppresses density from data-channel points. The plugin checks safe settings across NinjaLives, then checks the local interaction volume and object-type/tag selection before handing a body over.

> **Choose the interaction route deliberately**
> The FluidNinja Buoyancy component defaults to **FluidNinjaTracking**, which requires the compatible core and safe tracking settings above. **Wakes** selects the plugin's point-input route. Applying a preset preserves Interaction unless its Set Interaction flag is enabled. The separate Waves switch controls participation in the physical wave field. [S1] [S12]

03 · READING WATER

From a rendered surface to a physics force

The plugin's **Surface Mirror** is a top-down depth capture restricted to water primitives under relevant body footprints. It follows the actual displaced FluidNinja surface. During this capture, the added wave-height term is disabled and restored, so the capture records the original surface **W**. The new waves are sampled separately to allow each body's own contribution to be removed.



Three clocks, one coupled system

FluidNinja simulation tick · your Chaos time step · render frame rate

A readback is delayed data, even when polled as early as possible. The implementation exposes its age rather than assuming an exact three-frame delay.

Surface actors are discovered by tags. The plugin's default Surface Tags are `SplineSource`, `SeaSurface`, and `FluidNinjaBuoyancySurface`. Add the last tag to your pool or lake surface actor, or configure your own tag. These are **water surface actor tags**, distinct from the primitive tags used by tracking. Steep waterfall faces are rejected because a top-down height capture cannot represent them as ordinary supporting water. [S23]

GPU gather resamples water onto each body's grid. Compatible sources can be grouped, with limits on grid size, bodies, and update schedules. Completed asynchronous readbacks become immutable water patches for physics. Height-rate extrapolation is capped by `MaxExtrapolationSeconds` (0.08 s by default). It reduces apparent lag; it does not make the coupling instantaneous. [S4] [S15]

Converting FluidNinja flow into physical units

$$\text{current [cm/s]} = \text{buffer velocity} \times \text{SimSpeed} \times \text{cell size [cm]} \times \text{simulation ticks/s} \times \text{FlowScaleMultiplier}$$

The fixed Niagara tick takes priority over render FPS. For example, at 30 rendered frames/s a core configured for 60 Hz can take two simulation ticks each frame; halving the flow conversion because the display runs at 30 FPS would be wrong. The plugin's default Flow Scale Multiplier is **1**. Verify the actual simulation tick and cell size before calibrating current against measured motion. [S4] [S5]

Units to keep straight

Quantity	Authoring unit	Useful conversion
Position, draft, radius	cm	100 cm = 1 m
Volume	Hull cm^3 internally; readout litres	1,000,000 cm^3 = 1 m^3 = 1,000 L
Mass and density	kg and kg/m^3	Fresh water: 1 L displaced supports 1 kg
Force and impulse	Thrust in N; PokeAt in N·s	1 N = 100 kg·cm/s^2. Lower-level AddWaterImpulse uses kg·cm/s: 100 of those units = 1 N·s.

Quantity	Authoring unit	Useful conversion
Torque and power	N·m and W when explained here	Torque = force × arm; power = force × speed

04 · THE CUSTOM WAVE MODEL

How objects make waves and feel other waves

A body contributes **static head P**: the water-depth equivalent of its hull pressing into the surface. Motion of that footprint makes displacement wakes. Entry, drag, and slam contribute transient impulses. The core advances a free surface height **η** and vertical velocity **v** alongside FluidNinja's existing flow.

$$v_{next} \approx v - \Delta t\, g\, L(\eta + P + P_{dynamic}) - \Delta t\, \Gamma v$$
$$\eta_{next} = \eta + \Delta t\, v_{next}$$
$$\omega^2 = (gk + (\sigma/\rho)k^3)\tanh(kd)$$

Reading the equations. Gravity restores displaced water; **L** spreads that restoring influence across wavelengths; **Γ** represents damping. Wavenumber **k = 2π/λ** comes from wavelength λ; angular frequency **ω = 2πf**; **d** is depth; σ is surface tension. Longer waves and shorter ripples travel differently. In shallow water, long-wave speed approaches √(gd).

The implementation approximates this gravity–capillary dispersion relation with weighted samples from a mip hierarchy. It uses a symplectic Euler update and additional decay, shoreline, edge, and mean-level terms. It is **not an FFT ocean** and is not a 3D particle or volume fluid. The model is linear surface-wave physics with practical nonlinear damping and source limits. [S7]

Core step	What it does
Ninja_WaveParams	Emitter update sizes grids and targets, maps the scrolling simulation, and supplies parameters.
Wave Prepare	After vendor Pressure: carries prior waves with current and builds masked operator data.
Wave Step	Advances height/speed; applies depth, damping, breaking, and foam sources.
Wave Aux	Exports mask, depth, drift, and foam for the plugin.
Wave Export	After vendor pressure export: preserves pressure/divergence and writes wave/head channels.

The buffer contract

Data	Channels
Wave source texture	RGBA32F, width × (height+1). R = static head P in cm; G = impulse head integral in cm·s; B = erase flag; A = pressing hull speed through water in cm/s. Extra header row stores mapping/serial.
PressureDivergenceBuffer with waves	RGBA16F. R/G retain original pressure/divergence. B = η + P, the drawn wave height. A = P.
Wave auxiliary target	Wet mask, depth, and drift allow the plugin's own fields to use the same environment.
WaveFoamBuffer	Separate R16F foam F from 0 to 1. It does not feed density, flow, or buoyancy.

At equilibrium under a stationary hull, η tends toward −P, so the drawn term **B = η + P** tends toward zero instead of leaving a permanent visual hole. Moving that pressure footprint creates traveling waves. Impulse sources are spread spatially, constrained by the forces actually transmitted, and scaled by `WaveImpulseScale`, currently **0.35**. This is calibrated coupling, not an exact water-entry energy budget. [S5] [S8] [S16]

05 · FEEDBACK STABILITY

Why a body does not ride its own delayed wave

If a body presses water down, then feels that response several frames later as a new disturbance, it can feed energy back into its own motion. Each participating body therefore has a local **own-wave field**: a copy of the wave model driven only by its own sources.

All waves B
This body + other bodies
Shared core simulation

−

Own field B_self
This body's sources only
Same mask, depth and drift

=

Other-body waves
Safe input to buoyancy
Added to original surface W

The body's wave resistance is restored in physics
Radiation damping and added mass act during the current Chaos step.

Self subtraction removes delayed self-feedback. It does not remove waves generated by other bodies.

$$\text{water height felt by a body} = W + \kappa\,(B - B_{\text{self}})$$

$$\kappa = \text{project Wave Feel Scale} \times \text{body Wave Feel Scale}$$

Radiation damping represents energy carried away by the body's own waves. **Added mass** represents water accelerated along with the body. Both affect heave, roll, and pitch. The current source estimates motion frequency and uses frequency-dependent damping, with a natural-frequency fallback. Added mass works through Stable Steps. The rest-damping boost is **1** in the inspected implementation. [S3] [S6]

> **A quality tradeoff that affects behavior**
> A body without a usable own field can still make waves, but does not feel the added wave field. It continues to float on original FluidNinja water. Distance, tile fit, and capacity can cause this fallback. Check it when a distant float looks less responsive.

Foam and choppiness

Interface 4 generates foam from breaking, moving waterline shear, and impacts. Current carries it; exponential decay fades it. With `FoamDecay = 0.7`, the e-folding time is about 1.43 s: roughly 37% remains after that time, so it is not literally gone in 1.5 seconds. Foam changes appearance only. Separate Niagara whitewater emitters do not automatically consume this foam texture.

`WaveChoppy` is an optional material static switch, off by default. Its horizontal displacement sharpens crests visually. The physical queries remain based on the height-field model; treat choppiness as an appearance control.

06 · FORCES AND BALANCE

The physics that makes an object float

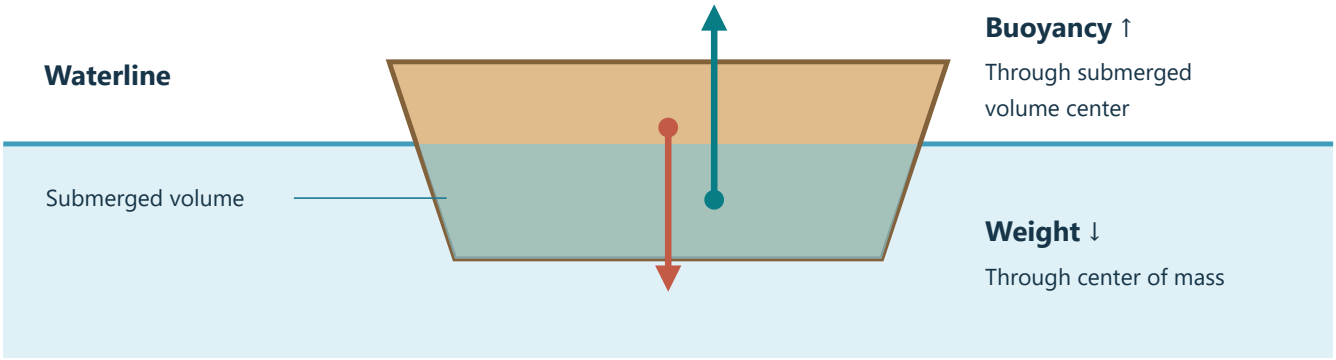
In calm water, a floating body displaces its own mass of water. Shape determines where that water is displaced; mass determines how much is needed. The center of buoyancy is the center of the submerged volume. Forces away from the center of mass produce torque, so a boat can pitch, roll, or right itself.

$$F_{buoyancy} = \rho_{water}\, g\, V_{submerged}$$

$$\text{At rest: } V_{submerged} = \text{mass} / \rho_{water}$$

$$\text{Torque} = (\text{force point} - \text{center of mass}) \times \text{force}$$

Keep `BuoyancyMultiplier = 1` for this physical relationship. If mass exceeds water density times the full supported volume, the object sinks. A hollow boat can float even though its construction material is denser than water: its sealed outer envelope displaces water while its total mass is much smaller than a solid block of the same size. Flooding and water entering the interior are not simulated. [S2] [Archimedes]



Schematic force points are offset for clarity. At a stable, level equilibrium their vertical lines of action align; a disturbance produces restoring or overturning torque.

Three hull representations

Hull model	Calculation	Use it for
Triangles	Subdivides simple collision, clips faces at the waterline, integrates hydrostatic pressure, drag, and slam over wet faces.	Boats and shaped hulls; logs when surface fidelity matters.
Voxels	Fills collision with cubes and integrates their wet portions.	Crates, blocks, and approximate debris. Voxel Count trades detail for cost.
Spheres	Uses exact submerged spherical caps for pontoons; projected/wetted geometry for resistance.	Balls and simple support points. Sphere rows/rings are approximations, not unioned solids.

The physics hull comes from **simple collision**, not the beautiful render mesh. Avoid overlapping collision pieces: they can double-count volume, lift, and drag. A closed boat envelope should contain the volume it keeps dry. A thin shell that only follows wood thickness will displace too little; several overlapping boxes can displace too much.

Drag and impact

Body-point velocity includes translation and rotation: $\mathbf{v_{point} = v_{COM} + \omega \times r}$. Subtract local water velocity to get motion through water. The plugin uses pressure and suction terms with linear and quadratic speed coefficients, angle falloff, ITTC skin friction, and slam on growing wet area. These coefficients are tuning controls for the implemented face model; they are not a single universal drag coefficient copied from a textbook. The familiar physical scale is $\mathbf{\frac{1}{2}\rho C_D A v^2}$. [S2] [Drag equation]

A heavier object does not automatically fall faster. Ignoring air resistance, entry speed from height H is $\mathbf{\sqrt{2gH}}$. It does carry more momentum $\mathbf{mv}$ and energy $\mathbf{\frac{1}{2}mv^2}$. Slam, entry power, visible splash, and traveling wave amplitude are related but separately modeled.

07 · STABLE BEHAVIOR

Mass, draft, and stability are different controls

Control	Meaning	Choose it when
Body Mass	Uses the primitive's kg. Solves the floating depth from that mass.	You know the actual object's weight or want it to sink realistically.
Rest Draft	At setup, sets mass equal to displaced water at this depth and Rest Attitude.	You are authoring a desired waterline. The primitive's previous mass is replaced.
Preset Density	Editor preset computes mass from collision volume × density, then uses Body Mass.	You want similarly modeled objects of one material to scale in weight.
Center Of Mass Offset	Moves the mass center and changes restoring moments.	The waterline is correct but the object rolls or tips unrealistically.
Inertia Scale	Changes resistance to angular acceleration.	The mass distribution of a hollow body differs from its collision solid.

Do not repair bad geometry with lift gain. First check units, scale, collision volume, mass mode, and draft. Then adjust center of mass and the collision envelope. Drag controls how motion settles; it should not be the main way to conceal a statically unstable boat.

The small-angle stability test

$$GM = KB + I_{waterplane} / V_{submerged} - KG$$

GM is metacentric height: a small-angle measure of whether a tilted body tends to return upright. KB is height of the buoyancy center above the hull bottom; KG is height of the mass center; I is the waterplane's second moment of area about the roll or pitch axis. Positive GM suggests restoring torque near that attitude; negative GM suggests tipping. It does not predict every large-angle capsize or collision. `Analyze Hull` reports roll and pitch stability for the modeled hull. [S2]

Why light bodies need Stable Steps

A light ball can gain a large amount of buoyant force after a tiny vertical movement. Its effective water spring is stiff compared with its mass. An explicit force step that is too long can overshoot and add energy. The plugin's `Stable Steps` handles the unresolved part of stiffness and drag implicitly and includes added mass, rather than demanding one global frame rate for every object.

$$\text{For intuition: stiffness } K \approx \rho g A_{waterplane}$$

$$\text{Natural period } T \approx 2\pi \sqrt{((m + \text{added mass}) / K)}$$

Small mass and large waterplane area give a short period. A physics step must resolve that period. Leave Stable Steps on for ordinary use; compare `fnb.Buoyancy.Implicit 0` and `1` only as a diagnostic, then restore `-1` to use project policy. Test the intended frame rate and physics settings in your project; a particular asynchronous physics rate is not a universal prerequisite. [S3] [S5]

> **Tune in this order**
> Collision and scale → physical mass or intended draft → balance and inertia → drag and current → wave coupling → splash, foam, and sound. Recheck after changing hull resolution because the discrete force and splash samples change.

08 · IMPLEMENTATION

Create the first floating object

Use a small calm test area before adding a river, waves, or character interactions. These steps describe the current editor workflow; they do not modify your project automatically.

1. **Validate the water setup.** Enable the compiled FluidNinja Buoyancy plugin and the matching modified FluidNinja core. Run **Tools** → **FluidNinja Buoyancy: Check Setup** or `fnb.CheckSetup`. Read each finding, especially surface discovery, core interface, buffers, and interaction policy.
2. **Configure shared settings.** Open **Project Settings** → **Game** → **FluidNinja Buoyancy**. Start with the plugin defaults, including fresh-water density 1000 kg/m³, Flow Scale Multiplier 1, and Stable Steps on.
3. **Identify the water surface.** Tag its actor using a configured Surface Tag. Confirm the material uses the right FluidNinja velocity/pressure buffers and simulation mapping. A mesh merely looking blue is not enough.
4. **Prepare a body.** Create an Actor Blueprint with a movable Static Mesh Component as root. Give it suitable simple collision, enable physics/collision, and use CCD for small fast drop objects. Prefer a mesh authored at its intended dimensions with unit component scale for the first test.
5. **Add one buoyancy component.** Add **FluidNinja Buoyancy** to the actor. If the floating primitive is not root, set `PrimitiveName` to its exact component name. Use one buoyancy component per simulated body.
6. **Apply a recipe.** Use a preset through the component's Preset panel, or right-click the placed actor → FluidNinja Buoyancy → Apply Preset. Enable Show Plugin Content to find `/FluidNinjaBuoyancy/Presets`. The actor-level tool can add a buoyancy component when one is missing.
7. **Set mass and starting behavior.** Use Body Mass for measured kg or density presets, Rest Draft for a chosen waterline. Leave Settle On Start on for an already floating object; turn it off for a drop. Keep Buoyancy Multiplier and Flow Coupling at 1 for the initial physical baseline.
8. **Analyze and test.** Click Analyze Hull. Read volume, mass, draft, and GM. In Play use `fnb.Buoyancy 2` and Get Readout. Check that a settled fresh-water floater displaces about one litre per kilogram, then test a gentle push and a small drop.
9. **Add feedback and presentation.** Confirm Waves is enabled for the body and simulation. Choose Wakes or safe tracking deliberately. Add the wetness material function and configure splash/sounds only after the body behaves correctly.

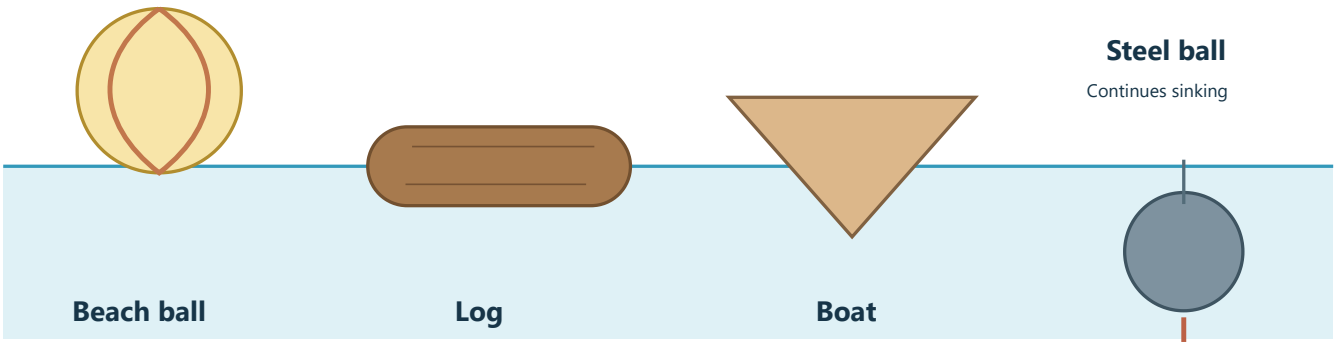
> **Apply hull presets before Play**
> Preset application writes properties and mass. It does not promise to rebuild an already initialized runtime hull. Make structural changes in the editor and restart Play. Use **Apply Buoyancy Preset To Component** when you need to target a particular existing component. [S12]

When gameplay teleports a floating actor, call `MarkTeleported()` after moving it. This lets the wave system erase the old footprint and reset the own field instead of treating the move as a violent water displacement.

09 · FOUR WORKED OBJECTS

Choose values from size and mass

These are calculated reference objects, not universal material prescriptions or newly tested prefabs. Assume fresh water at **1,000 kg/m³**, unit component scale, accurate simple collision, Buoyancy Multiplier 1, and calm water without contacts. Force examples use **g = 9.8 m/s²**, consistent with a common Unreal world gravity; the wave core uses about 981 cm/s². Use the target world's actual gravity when comparing force.



Behavior schematic, not a shared scale drawing. Volume fraction is different from fraction of diameter or height.

Reference object	Geometry and mass	Expected calm result
Inflated ball	60 cm diameter; 0.9048 kg, equivalent bulk density 8 kg/m³	0.9048 L displaced; 3.154 cm draft; only 0.8% of volume submerged.
Pine-like log	3 m long × 40 cm diameter; density 500 kg/m³ → 188.50 kg	188.50 L displaced; 20 cm draft; half the cylinder volume submerged.
Teaching V-bottom boat	3 m × 1.2 m × 0.5 m closed triangular prism; 240 kg total	240 L displaced; 25.820 cm draft; 24.180 cm freeboard.
Solid steel wrecking ball	80 cm diameter; density 7,850 kg/m³ → 2,104.45 kg	Full capacity only 268.08 kg of fresh water; it sinks.

> **Use measured mass when available**
> Wood species and moisture vary; beach balls differ in size, shell, and inflation; boats carry payload; wrecking balls may use different alloys or construction. These explicit assumptions make the calculations accurate for the chosen model. They do not identify every real object of that name.

Shared baseline for the recipes

Start with `FlowCoupling=1`, `BuoyancyMultiplier=1`, `InertiaScale=(1,1,1)`, body and project `WaveFeelScale=1`, and Stable Steps on. Retain pressure drag 0.3 linear / 1 quadratic, suction 0.3 / 0.5, viscosity multiplier 1, slam multiplier 0.5, and air coefficient 0.8 until you have a measured target. These are implementation starting values, not certified drag measurements for each object.

For these controlled recipes, explicitly choose `Interaction=Wakes` so the per-body Wake switch controls the point-wake route. This is an example choice, not the component default. The distinct Waves switch controls physical wave generation/response; Wake off does not turn Waves off. You can later compare safe FluidNinja Tracking under the same conditions.

10 · LIGHT AND NATURAL OBJECTS

Beach ball and log recipes

A 60 cm beach ball

Use a sphere centered at the body origin, radius **30 cm**, with matching simple collision. A One Sphere preset fits radius from the average of the scaled bounds' three half-extents, so do not apply it to a stretched ellipsoid and expect an exact fit.

A 3 m pine-like log

Use a closed cylinder along local X, length **300 cm** and radius **20 cm**. A sufficiently detailed closed convex approximation is suitable; a bounding box will have the wrong volume and waterline.

Setting	Beach ball	Log
Hull Model	Spheres	Triangles
Hull sampling	One pontoon at (0,0,0), Radius 30 cm, LiftWeight 1, DragWeight 1	HullResolutionCm 20; MaxHullTriangles 1000
Mass Mode and body kg	Body Mass; 0.904779 kg	Body Mass; 188.495559 kg
Rest Attitude	(0,0,0)	Zero for X-axis cylinder; Pitch 90° for a cylinder modeled along Z
COM and angular damping	COM centered; damping (0,0,0) baseline	COM centered; damping (0,0,0) baseline
Settle On Start	On for float test; off for drop	On for float test; off for drop
Sleep When Settled	Off while observing small wave response	On for production; off during response tests
Wake / Waves	Wake off; Waves on	Wake on; Waves on
SplashMinWatts	20 W, matching public BeachBall preset	20 W, matching public Log preset

The ball calculation

$$V_{full} = 4\pi R^3/3 = 113.097\ \text{L}$$
$$m = 8\ \text{kg/m}^3 \times 0.113097\ \text{m}^3 = 0.904779\ \text{kg}$$
$$V_{cap}(h) = \pi h^2(R - h/3) = m/\rho \rightarrow h = 3.15415\ \text{cm}$$

Draft h is measured from the sphere's bottom. The center sits about 26.846 cm above calm water. The shipped `BeachBall` preset uses density 8; its prose says approximately 0.8 kg at 60 cm, but the ideal sphere result is 0.905 kg. Collision approximation can explain a different applied mass. For a lighter actual ball, enter its measured kg; for example, **0.3 kg** at the same diameter gives about **1.80 cm** draft. Use the calculator for the exact chosen mass.

The log calculation

$$V = \pi R^2 L = \pi \times 0.2^2 \times 3 = 0.376991\ \text{m}^3$$
$$m = 500 \times V = 188.496\ \text{kg}$$
$$m\ /\ (\rho V) = 0.5 \rightarrow \text{horizontal cylinder draft} = R = 20\ \text{cm}$$

A round, uniform log has almost no preferred rotation about its long axis. Near-zero roll stability can be correct. Do not automatically lower its center of mass to make it resist rolling unless the modeled log actually has a weighted or asymmetric side.

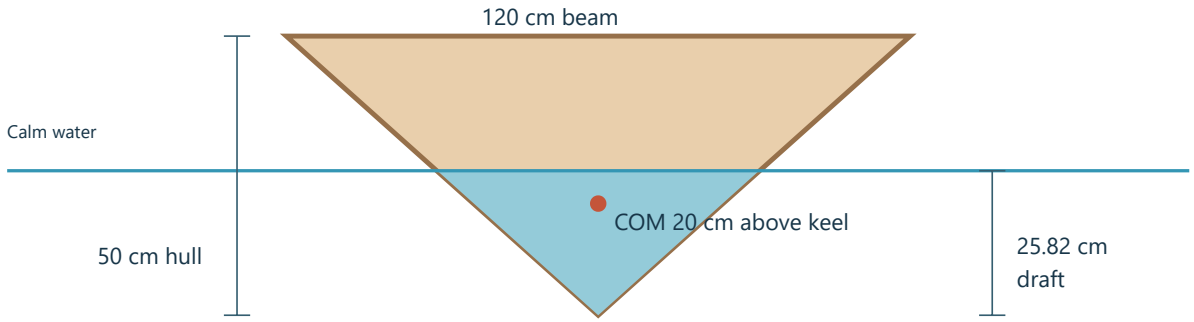
The public `Log` preset uses collision-derived triangles, 20 cm resolution, density 500, and Longest Axis Flat. An alternative for this exact ideal cylinder is **Rest Draft 20 cm**. The two approaches agree for ideal geometry; an approximate collision hull changes the mass or waterline produced by the preset. [S11]

11 · DISPLACEMENT AND SINKING

A small boat and a wrecking ball

A boat with an explicitly defined displacement hull

Use a **closed triangular prism** as a teaching hull: length 300 cm, beam 120 cm, keel-to-gunwale height 50 cm. At each end X = ±150 cm, the Y/Z cross-section vertices are (0,0), (−60,50), and (60,50). Close the top. The pivot is at the keel midpoint. This sealed envelope models excluded water; it does not simulate flooding.



Analytic reference hull at 240 kg in fresh water. A real tapered boat needs its own hull analysis.

$$V(d) = L\ B\ d^2\ /\ (2H)$$
$$d = \sqrt{(2H\ m\ /\ (\rho\ L\ B))} = 25.8199\ \text{cm}$$
$$\text{Full volume} = LBH/2 = 900\ \text{L}$$

Use **Triangles**, resolution **20–30 cm**, budget **1000**, Body Mass **240 kg**. Settle On Start on, Waves on, Wake on, propulsion off for the first hydrostatic test, and Splash Min Watts 300. Target a center of mass **20 cm above the keel**. If Chaos starts at the uniform prism centroid 33.333 cm above keel, COM offset Z = **−13.333 cm** achieves that at unit scale; verify the actual COM before copying the offset.

Ideal roll GM is about **22.0 cm** for that mass center. An extra centered 80 kg gives total mass 320 kg and draft **29.814 cm**, about 3.99 cm deeper. A rider placed to one side changes both load and balance. The flat-ended prism has unusually strong pitch stability; it is an explanatory hull, not a prediction for all dinghies.

> The public **BoatHull** preset chooses **13 cm Rest Draft** and 30 cm triangles. On this V-hull, 13 cm corresponds to only **60.84 kg**. A draft preset is not a universal boat mass. Use total known boat-and-payload kg for a real vessel.

An 80 cm solid steel wrecking ball

Use matching sphere geometry/collision, one pontoon at the origin with **radius 40 cm** and both weights 1. Set Body Mass to **2,104.448 kg** for the chosen steel density 7,850 kg/m³. Turn **Settle On Start off**, enable CCD, keep Waves on, Wake on, and use Splash Min Watts 20 as a starting visual floor.

$$V = 4\pi \times 0.4^3/3 = 0.268083\ \text{m}^3$$
$$\text{Mass} = 7{,}850 \times V = 2{,}104.448\ \text{kg}$$
$$\text{Maximum buoyancy} = 1{,}000 \times 9.8 \times V = 2{,}627.21\ \text{N}$$
$$\text{Weight} = 20{,}623.59\ \text{N} \rightarrow \text{net } 17{,}996.38\ \text{N downward before drag}$$

It still receives upward buoyancy; that force is too small to support it. Drag opposes sinking, while the bed or a crane constraint ultimately controls its motion. The buoyancy component does not create a cable or crane. A suspended wrecking ball needs an appropriate Chaos constraint as a separate system.

The shipped `WreckingBall` preset assumes iron at **7,800 kg/m³** and inherits settling on; turn settling off for a drop. Its density differs from this example's chosen steel value. Use the density of the material you intend, or enter a measured body mass directly. [S11]

12 · EXPLORE THE RELATIONSHIPS

Buoyancy calculator

This ideal-geometry calculator solves calm-water displacement. It is useful for catching a unit error or an impossible floating mass before opening the editor. It does not calculate waves, drag, collision contact, flooding, or capsize.

> Open the companion HTML guide to change the dimensions and mass interactively. The table below records the reference examples for this printable edition.

Shape	Volume formula	Draft relationship
Sphere	$4\pi R^3/3$	$\pi h^2(R - h/3) = m/\rho$
Horizontal cylinder	$\pi R^2 L$	$L[R^2 \operatorname{acos}((R-h)/R) - (R-h)\sqrt{(2Rh-h^2)}] = m/\rho$
Rectangular box	LWH	$h = m/(\rho LW)$
Closed V prism	$LBH/2$	$h = \sqrt{(2Hm/(\rho LB))}$

All formulas here use consistent SI units internally. The displayed draft is centimetres. The cylinder lies horizontal; the V-prism has straight sides and a level keel. If mass equals full water capacity, the ideal object is neutral when fully submerged, without a unique freeboard. If mass is greater, no floating draft exists.

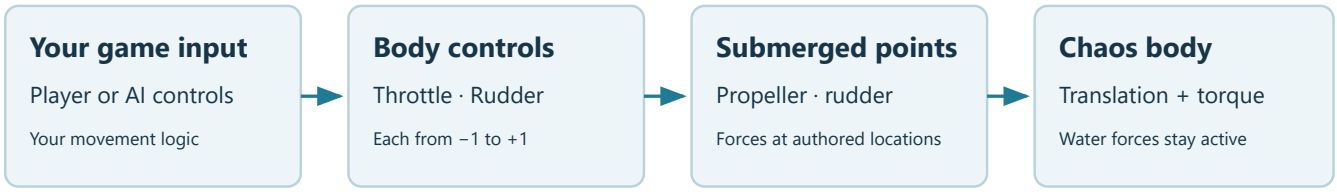
A change to try

Keep the log at 3 m × 40 cm. Its full displacement is about 377 L, so a 453.6 kg log of the same geometry sinks. Increasing weight while keeping dimensions fixed changes average density. A heavier log made of the same 500 kg/m³ wood needs a larger volume instead.

13 · GAMEPLAY INTEGRATION

Add propulsion and steering to your boat

The FluidNinja Buoyancy component includes a simple propeller-and-rudder model. It applies thrust and rudder force alongside the hull's water forces. Your Blueprint or C++ gameplay supplies `Throttle` and `Rudder`; the plugin does not require a boat data asset, seating system, or animation framework. [S1] [S18]



The component produces forces at two independently sampled water locations. Game controls are optional consumers of its public properties.

Propulsion property	Public source default	Meaning
Propulsion / Console Control	Off / On	Enable propulsion for a powered boat. Disable Console Control when gameplay owns the controls.
Thrust Location / Direction	(0,0,0) cm / +X	Application point and ahead direction in the body's frame.
Max Thrust / Prop Wash Speed	1500 N / 300 cm/s	Full-throttle push and signed flow contribution used at the rudder.
Rudder Location / Area	(0,0,0) cm / 900 cm^2	Force point and blade area. Area 0 disables rudder force.
Max Rudder Angle	35°	Angle at full left or right input.
Throttle / Rudder	0 / 0	Blueprint-writable inputs, clamped to [−1,1] for physics.

A concrete integration for the teaching boat

1. First complete the 240 kg V-hull's calm-water test with propulsion off. Orient the hull so the bow is local **+X** and right is **+Y**.
2. Before Play, enable Propulsion. As an **illustrative placement**, put both Thrust Location and Rudder Location at **(−140,0,10) cm** in this hull's keel-origin frame. They sit near the stern and below its calculated 25.82 cm waterline. For your mesh, use its actual geometry, pivot, and waterline instead.
3. Keep Thrust Direction +X. Start with Throttle and Rudder at zero. Map a forward/reverse input to Throttle and a left/right input to Rudder; reset them to zero when input ends. Use Console Control off for this gameplay path.
4. Test a small throttle, then steering while moving. Verify the forces disappear when their own points leave the water. A dry rudder does not steer just because the hull is floating.

$$F_{thrust} = Throttle \times MaxThrustNewtons \times direction$$

$$U = dot(v_{body\ point} - v_{current}, forward) + Throttle \times PropWashSpeed$$

$$C_L = clamp(2\pi \times rudder\ angle\ in\ radians, -1.2, 1.2)$$

$$F_{rudder} = -right \times \tfrac{1}{2}\rho\ A\ C_L\ U\ |U|$$

$$Torque = (force\ point - center\ of\ mass) \times force$$

The rudder uses the point's linear and angular body motion, horizontal current scaled by Flow Coupling, and propeller wash. Convert area to m^2 and speed to m/s if evaluating this equation in SI units. At the default 1500 N maximum, Throttle 0.2 commands **300 N** when the propeller point is wet. For the 240 kg example, F/m is **1.25 m/s^2** before drag, added-mass response, and other forces. It is a force estimate, not a promised acceleration or top speed. [S2] [S18]

> **The rudder assumes body +X forward**
> Thrust Direction can change the propeller direction, but the built-in rudder still uses the body's X/Y axes. Rotating that one property does not rotate the rudder model. Also, the default force points are at the origin: move them to meaningful locations or they may be dry or provide little turning leverage.

For console testing, enable Console Control and use `fnb.Boat.Throttle 0.2` and `fnb.Boat.Rudder 0.25`, then set both to 0. Changes affect all bodies with Propulsion and Console Control enabled, not only the selected boat. [S4]

Seats, boarding, animation-driven oars, passenger loading, and network authority belong to your game. Include payload in the total mass and balance used by your hull setup, and test changes against the intended waterline. The plugin's propeller-and-rudder controls do not automatically add a passenger's mass or implement those gameplay systems.

14 · WATER SETUP AND QUALITY

Configure the water that objects use

There are four scopes of control: project settings choose shared defaults, a Wave Sim component describes one simulation, each buoyancy component describes one body, and material parameters control appearance. Confusing those scopes is a common cause of inconsistent behavior.

Setting	Current source default	Practical use
Capture Width / Resolution	8,000 cm / 1024	About 7.81 cm base texel size across an 80 m square. Wider coverage at fixed resolution is coarser.
GPU Gather / grid limit	On / 128 across per body	Batch water sampling; large footprints may be sampled more coarsely.
Max Extrapolation	0.08 s	Limits height prediction from old readbacks.

Setting	Current source default	Practical use
Water Density / Stable Steps	1000 kg/m³ / On	Fresh-water support and handling of stiff light-body response.
Flow Scale / Max Flow	1 / 600 cm/s	Converts and caps sampled current. Keep this separate from per-body Flow Coupling.
Wave Damping / Breaking Slope	0.02 s^{-1} / 1	Decay and suppression of excessively steep features.
Wave Impulse / Added Mass scale	0.35 / 1	Entry-wave coupling and inertia of entrained water.
Foam Decay	0.7 s^{-1}	Trail lifetime; lower persists longer. Does not change lift.

Give each simulation a depth and edge policy

Add **FluidNinja Wave Sim** to a NinjaLive actor for settings that travel with the level. Its component takes precedence over a matching name-based entry in project `WaveSims`. Set one authority deliberately rather than editing both. [S14]

Per-simulation property	Default	Meaning
Depth Cm	200	Constant depth when Depth From Bed is off. Use the actual modeled depth.
Depth From Bed	Off	On uses the core's bed field; missing bed falls back to project WaveDepthCm, not this component's DepthCm.
Walled	Off	Off absorbs outgoing waves at window edges; on reflects them with wall absorption.
Wall Absorption / Band	0.4 s^{-1} / 50 cm	Energy loss near the simulation-window wall. It does not create physical collider walls.
Scale Down Override	0	0 follows quality; 1/2/4 chooses full/half/quarter wave-grid width.
Importance	1	Higher priority when only the nearest weighted simulations receive active waves.

Choose depth from the water body you intend to model. For example, a 2 m test pool can use Depth Cm 200 with Depth From Bed off; choose reflective or absorbing edges according to the test. This is an illustrative setup, not an automatic measurement of the scene. A height field that sees the water mesh rather than the bed produces a wrong depth; use a known constant depth for that setup.

Water outside the capture: the generic far-field path uses a `PhysicsVolume` with **Water Volume** enabled. It takes the volume bounding box's top as a flat water height and checks horizontal coverage. Align that top with the intended still-water surface. This supplies simpler fallback coverage; it does not extend the GPU simulation or reproduce local waves outside its window. [S4]

Budget behavior

`sg.WaterQuality -1` follows Effects quality; 0–4 selects a tier. For the current plugin tier config, Low uses scale-down 4 and one active wave sim; Medium 2 and two; High 1 and four; Epic 1 with no active-sim cap. Own-field reach is also reduced at lower tiers. Inspect `stat FluidNinjaBuoyancy`, `fnb.Memory`, and `fnb.Waves.Memory` in the intended workload.

Halving grid width cuts wave Prepare/Step/Aux cells to one quarter, but export still writes the original pressure resolution. Do not promise a fourfold total GPU speedup. Multiple simulations have independent windows; their waves do not automatically propagate across each other's boundaries.

15 · VISIBLE INTERACTION

Waves, brush wakes, foam, and splashes

These effects share a scene, but use different data and controls. Tuning the wrong one can change current when you only meant to make whitewater more visible.

Path	Input and result	Controls
Physical wave field	Static hull head and force-derived impulses → propagating surface waves.	Waves, WaveFeelScale, WaveDisplacement, WaveImpulseScale, depth and damping.
Relative tracking	Selected primitive motion relative to water → FluidNinja painter interaction.	Interaction, tracking volume/filter/tag, relative switch, density suppression.
Point wakes	Waterline push and plunge contacts → Niagara Data Channel brush points.	Wake, WakeGain, WakeStrength, PlungeGain/Response, point reader and budgets.
Wave foam	Breaking, waterline shear, and impacts → independent visible foam field.	Foam enable/decay/source gains, WaveFoam material controls; interface 4.
Splash and audio	Entry/slam/bow activity → Niagara and sound instances.	Power floors, reference watts, strength ceilings, scale, intervals and budgets.

Entry and slam are now separate

The C++ force integrator collects growing wet area across the body before distributing its slam, with an inward-speed gate of 50 cm/s. A mass-limited force prevents a piecewise water-entry impulse from arbitrarily stopping more than its supported share of the body in one step.

$$G = \Sigma(dA_i/dt),\ Q = \Sigma((dA_i/dt)\ v_i),\ \bar{v} = Q/G$$

$$F_{slam} = \min[\text{SlamMultiplier}\ \rho\ Q\ \text{beam},\ (m\ \bar{v}/\Delta t)\ \min(2A_{participating}/A_{hull},\ 1)]$$

$$P_{entry} = m\ v_{down}{}^{3}\ /\ (2H_{body})$$

$$\text{Splash strength} \approx (\text{power}\ /\ \text{SplashReferenceWatts})^{1/3}$$

Entry power is a visual estimate of kinetic energy delivered over an entry time; it is not an extra physical force. The default reference is **70 W**; body and project ceilings normally cap strength at **12** before the final art scale. Twice the visible scale requires eight times the power before those caps. Splash Min Watts gates repeated slam splashes; setting it to zero does not disable entry splashes or bow spray. [S2] [S5]

For more white wake: first tune interface-4 foam or its material strength. Point-wake gains inject into FluidNinja's flow/density interaction and can alter river speed. Neither a symmetric burst nor a gain below 1 guarantees conservation in the full nonlinear flow setup. Compare mean current before and after adding interacting bodies when calibrating this route.

Hull wetness and sound

Wet Line writes four contiguous Custom Primitive Data slots starting at `WetLineDataIndex` (default 0). The hull material needs `MF_FNB_HullWetness` or equivalent water-plane CPD integration. Enabling the property alone does not change an arbitrary material. Reserve those slots to avoid collisions with other material features. Drying Speed defaults to 1.5 cm/s. [S1] [S18]

The plugin exposes configurable splash and sound assets. Assign the assets appropriate to your project; slap, wash, creak, and optional drip have separate roles. Check visibility distance and global effect budgets when physics is correct but spray is missing.

16 · EXTENDING THE SYSTEM

Queries, swimmers, and integration points

Gameplay should consume the plugin's water API rather than independently decode GPU textures. This preserves the chosen source mapping, readback age, and wave convention.

Interface	What to use it for
Get Water At Location	Request/sample water at a point. Call each frame for a moving query, and handle a missing or delayed answer.
Filter Drift Current	Smooth noisy current and dead-band small speeds. Source defaults are 0.5 s smoothing and 10 cm/s dead band.
Get Swimmer Water	Read the registered swimmer's shared water sample and filtered drift current.
Poke At	Apply a localized wave impulse for a footstep or paddle. Strength is N·s; visual splash and sound remain separate.

Interface	What to use it for
Get Readout / Get Status	Inspect a buoyant body's diagnostic physics state or human-readable status.
On Slam / On Water Contact Changed	Trigger gameplay from slam power or entering/leaving water.
Mark Teleported	Reset wave bookkeeping after a manual body relocation.

A robust query flow

```
Each game update
  Get Water At Location(position) → success, sample
  If no water: use the gameplay dry state
  If water:
    use SurfaceZ and Depth for the surface relationship
    check bFromPatch to distinguish capture from far-field height
    check bHasCurrent and bHasWaves before trusting those fields
    inspect AgeSeconds, then filter Current for movement
  Ask again next update as the character moves
```

For swimmers, `SurfaceZ = StillZ + η`, with `η = pressure.B − pressure.A`. `Depth = SurfaceZ − query.Z` is negative above water. Current is horizontal; its Z component is zero. The returned normal comes from the wave slope. A query can use an existing body's capture patch, otherwise a known far-field water surface. An arbitrary point query does not create a fresh whole-scene capture by itself. [S13]

The plugin supports swimmer registration and a **FluidNinja Swimmer** component for body-part contacts, displacement, and wave impulses. The component registers its mesh and exposes `GetWater`, `GetDriftCurrent`, and stroke controls. Your movement code must consume the water sample and current; registering a swimmer does not create a swimming locomotion system. [S21] [S22]

For a custom movement system, implement `IFluidNinjaSwimmerProvider::GetSwimmerPose`, a Blueprint-native interface event, or derive a swimmer component in C++. A swimmer profile configures its body-part contacts. The plain C++ virtual state hooks are not automatically Blueprint override events. Wind is supplied through the plugin's world-space `Wind` setting in cm/s, with zero wind by default; your weather system can provide that input. [S24] [S25] [S5]

> **Readout is diagnostic, not the final net force**
> `ForceN` reports raw hydrodynamic integration. The callback subsequently adjusts forces for radiation damping, stable steps, and added mass. Gravity and character contacts are separate. Do not infer exact body acceleration by dividing this readout by mass.

Not every property marked editable has a Blueprint setter. The explicit runtime controls include Throttle, Rudder, Interaction, Waves, and Wave Feel Scale; use the declared APIs. Hull rebuilding and arbitrary live preset swapping are not established by these headers.

17 · MATHEMATICAL REFERENCE

What the force integrator computes

This section links the simple Archimedes explanation to the implementation. Use consistent units when evaluating formulas. The public density is kg/m³; the integrator converts it to kg/cm³, and converts force and power back to N and W for readouts.

Triangle pressure and center of pressure

$$F_{piece} = -n\ \rho\ g\ h\ A \times \text{BuoyancyMultiplier}$$
$$D = d_1 + d_2 + d_3$$
$$r_{pressure} = [(D+d_1)r_1 + (D+d_2)r_2 + (D+d_3)r_3] \ / \ (4D)$$

A triangle is clipped at the sampled waterline. For each wet piece, n is outward normal, A is area, h is mean depth, and d_i are vertex depths. Pressure grows linearly with depth. The depth-weighted point matters: applying every force at the triangle centroid can give the wrong righting torque even when total lift is correct. [S2]

Triangle pressure drag and skin friction

$$s = |v_{relative}| \ / \ (1\ \text{m/s}),\ q_0 = \tfrac{1}{2}\rho(1\ \text{m/s})^2$$
$$c = n \cdot \text{normalize}(v_{relative})$$
$$\text{For } c > 0\text{: } F_{pressure} = -n\ q_0\ A\ (C_{p1}s + C_{p2}s^2)c^{f_p}$$
$$\text{For } c \le 0\text{: } F_{suction} = +n\ q_0\ A\ (C_{s1}s + C_{s2}s^2)(-c)^{f_s}$$
$$C_{friction} = 0.075 \ / \ (\log_{10} \text{Re} - 2)^2$$

Skin friction opposes tangential velocity with a ½ρC A v² scale. Reynolds number uses hull length and viscosity, with a minimum of 100,000 in this implementation. Spheres and voxels use a projected-section drag approximation rather than the triangle suction/falloff model. Their geometry scale means a textbook sphere drag coefficient is not automatically a calibrated terminal-speed setting here.

Spheres and voxels

$$\text{Sphere h} = \text{clamp}(\text{water Z} - \text{center Z} + \text{R}, 0, 2\text{R})$$
$$V_{cap} = \pi h^2(3R - h)/3$$
$$\text{Voxel wet fraction} = \text{clamp}((\text{water Z} - \text{center Z})/\text{cell size} + \tfrac{1}{2}, 0, 1)$$

A sphere's lift is $\rho g V_{cap}$ times LiftWeight and the buoyancy multiplier. Explicit pontoons are summed, not Boolean-unioned. Voxels share the collision volume across retained interior samples; their simple vertical wet fraction is not exact rotated cube clipping. The triangle surface is still retained for wave-source displacement, so a sphere/voxel lift model should remain consistent with collision geometry.

Implicit treatment of unresolved response

$$\Delta v = \Delta t\,(F + W + A\,a_{water} - \theta\,\Delta t\,K_{excess}\,v) \div (m + A + \Delta t\,C_{excess} + \theta\,\Delta t^2\,K_{excess})$$

This heave expression shows Stable Steps with excess stiffness K, excess damping C, added mass A, signed gravitational force W (negative in upward-positive Z), and filtered water acceleration. θ defaults to 1. The actual force is rewritten so Chaos reaches this velocity change using its own body mass. Analogous inertia/torque handling applies to rotation. Stiffness and damping already resolved by the step are left explicit. Stability can include numerical damping; it is not a guarantee of identical motion at every frame rate. [S2] [S3]

The sampled height rate helps predict height but is not used as vertical drag-current velocity; that current remains horizontal. Added mass has a separate filtered water-acceleration estimate. This distinction prevents abrupt visual height changes from becoming spurious vertical drag kicks.

18 · VERIFICATION AND TROUBLESHOOTING

Know when the setup is correct

Validate static support before evaluating waves and spray. A good visual splash does not prove a physically meaningful mass or waterline. Build a small test level with a tagged water surface, one compatible NinjaLive, and one object from these recipes. Keep its dimensions, mass, gravity, water depth, quality tier, and source versions with the results so another person can repeat the test.

Test	What to observe	What a failure suggests
Hull analysis	Correct dimensions, full volume, mass mode, solved draft, and plausible GM.	Scale error, overlap, wrong primitive, thin-shell collision, or wrong mass authority.
Calm settle	Displaced litres ≈ mass kg in fresh water with lift gain 1.	Wrong body mass, collisions holding it up, waves/current still active, or coarse hull.
Small push	Damped motion and expected righting/rolling behavior.	Bad COM, excessive damping, unresolved stiffness, or mistaken cylinder roll expectation.
Current drift	Body approaches water motion; river does not progressively accelerate from floaters.	Tracking density feedback, wrong flow conversion, or overdriven point wakes.

Test	What to observe	What a failure suggests
Controlled drop	Settle disabled; measured entry speed, EntryWatts/SlamWatts, wave and spray paths.	Settling canceled the fall, missing water readback, effects budget, or core mismatch.
Neighbor wave	Drop A; watch B respond; check B has its own field and Waves/feel enabled.	Separate sim windows, stale patches, quality/atlas fallback, or wave controls off.
Walk away and return	Sleep/LOD/re-entry behavior is acceptable at intended quality.	Body or surface not rediscovered, limits too low, or assumptions about distant wave response.

Current diagnostic commands

```
fnb.CheckSetup                editor setup audit
fnb.Buoyancy 2                physics and water debug view
fnb.Buoyancy -1               restore project policy
fnb.TestHUD 1                 enable developer panel in PIE
fnb.Test help                 list supported tests
fnb.Test run settle           controlled settling test
fnb.Test run drop             controlled drop test
fnb.Test run gather           compare sampling paths
fnb.Test run ripples          wave test suite
stat FluidNinjaBuoyancy       runtime counters
fnb.Memory                   readback / runtime memory
fnb.Waves.Memory             wave resources
```

Debug overrides `fnb.Buoyancy.Wake`, `.Gather`, and `.Implicit` accept −1 for project policy, 0 off, 1 on. The test HUD defaults off and belongs to the optional developer module. Use `fnb.Test help` to inspect the tests available in your build. Do not use unsafe tracking force-handover as a normal fix. [S20]

> **Keep evidence for your installed combination**
> Run the relevant tests against the exact core, plugin, collision assets, and settings you will ship. Record test reports and repeat important measurements after changing them. Source explanations and the calculated examples in this guide are not runtime pass results.

Known model and product limits

The capture is a top-down height representation. It cannot resolve overhanging water sheets, overturning breakers, full underwater 3D flow, or physically conserved filling/flooding of a hull. Readback remains delayed. Source limits and quality fallbacks affect distant response. Include wave amplitude, long-term mean water level, and closed-basin behavior in tests when those matter to your game.

Test your intended platform, rendering path, physics settings, packaged build, and network authority explicitly. A dedicated server has no rendered Surface Mirror; simpler known water coverage cannot reproduce every GPU wave or current detail. The force and query APIs do not by themselves define multiplayer authority or replication. [S4]

19 · CORE VERSIONS AND MAINTENANCE

Keep the plugin and core compatible

The plugin's editable HLSL and editor toolchain live under `Tools/CorePatch`. The generated Niagara and material packages live under FluidNinja content. A vendor update can replace those modifications, so upgrading FluidNinja is a compatibility task rather than a content copy alone.

Core interface	Capability	Compatibility meaning
Relative tracking only	Painter changes, no versioned wave interface	Insufficient for the current wave contract.
2	Versioned waves, corrected breaking/edge behavior	Current minimum wave interface; full-resolution wave grid, no new foam.
3	Independent wave-grid scale-down	Supports 1/2/4 grid width choices.
4	Foam field plus matching material foam/choppy additions	Current toolchain capability; needed for independent wave foam.

Check the installed core's interface and build values through the setup checker and the core tool's `verify` report. Record those results with the plugin source revision and matching material capability. A plugin version number such as 1.0.1 is not a substitute for checking the actual core assets and buffers. [S4] [S9]

Safe implementation sequence

1. Record the plugin source revision, FluidNinja version, current core interface/build, and known-good test reports. Preserve a recoverable copy of the affected assets before patching.
2. Run the editor setup checker, then close the editor before the core tool's commandlet `verify` or patch workflow. Require its report to identify the core and finish `RESULT PASS`; read warnings about older capabilities.
3. For a new vendor version, first port the relative-tracking graph changes. The tool's `tracking` step is **not implemented**; it cannot automatically turn a pristine vendor core into the required safe base.
4. From the supported relative-tracking base, follow the tool's `core material verify` workflow. The core step refuses a core already carrying waves; use the documented reapplication/backup procedure rather than blindly applying twice.
5. Perform material changes in a fresh commandlet/editor process before PIE, following the patch tool's workflow.
6. Open the intended level, validate surfaces and sim assignments, then run settling, drift, entry, neighbor-wave, and intended-quality tests. Keep the compile and test reports with your recorded version information.

The patch changes `NinjaLiveCore`, `Core/Functions/PainterSourceData`, `Core/Modules/Ninja_Painter`, the generated `Core/Modules/Waves/*`, and `OutputMaterials/Base/MF_NinjaOutput_BaseFunction_SURFACE`. The tracking and wave layers have different authoring steps. Read the plugin's `Tools/CorePatch/README.md` and `Tools/CorePatch/TRACKING.md` before maintaining them.

Presets: Keep custom presets in project content; `Tools/make_presets.py` rewrites the shipped library. Customize a copy or update its generator deliberately.

20 · SEARCHABLE IMPLEMENTATION REFERENCE

Property reference

The tables below are generated from the current public editable declarations: **184 properties** across the body component, project settings, per-simulation component, and preset data asset. Names are exact C++ identifiers so you can search source or Details. Unreal generally displays them as spaced labels and removes the boolean b prefix.

Header defaults are not saved instance values. Constructors, configuration, presets, Blueprints, and actor instances can replace them. "Constructor / empty" means no initializer in that declaration. Edit conditions can hide or disable a property. Some descriptions summarize source comments; use the earlier chapters for model limits and behavior.

> **Separate defaults from recipes**
> The component starts with Interaction = FluidNinjaTracking, Mass Mode = Rest Draft, and Rest Draft = 15 cm. The recipes explicitly choose their mass, hull, and interaction settings. Apply those choices deliberately and inspect the resulting component rather than assuming a preset changes every property. [S1] [S12]

Body component / Hull [S1]

Exact property	Header default	Meaning
`PrimitiveName`	`Constructor / empty`	The floating primitive; None uses the actor's root.
`HullModel`	`EFluidNinjaHullModel :: Triangles`	How the body meets the water: the hull surface (boats), voxels (logs, crates) or spheres (pontoons: balls, small props).
`HullResolutionCm` cm	`40.f`	The hull is rebuilt from the simple collision as an even shell of triangles about this size, so the water line is followed across big faces (coarser if Max Hull Triangles is reached).
`MaxHullTriangles`	`1000`	The most triangles the hull may have; past it the shell is made coarser than Hull Resolution. More follows the water line more closely and costs more every physics step. The farther levels of detail get a quarter and a sixteenth of it.
`LodDistancesCm` cm	`FVector2D(3000.0, 8000.0)`	Distances from the camera.
`VoxelSizeCm` cm	`0.f`	Voxels model: cube size (0: about Voxel Count of them fill the hull).
`VoxelCount`	`48`	Voxels model: about how many cubes fill the hull when Voxel Size is 0. More follows the shape more closely and costs more.
`Pontoons`	`Constructor / empty`	Explicit sphere centers, radii, LiftWeight and DragWeight. Positions and radii are scaled body-frame centimeters. Empty array uses generated support rows.
`bSleepWhenSettled`	`true`	Once settled (barely moving for two seconds), out of sight and farther than Sleep Distance from the camera, the body sleeps and the water leaves it alone until it is seen, comes near, or something hits it.

Exact property	Header default	Meaning
`SleepDistanceCm` cm	`6000.f`	A settled body sleeps only when at least this far from the camera (and out of sight); it wakes when it comes nearer.

Body component / Floating [S1]

Exact property	Header default	Meaning
`MassMode`	`EFluidNinjaBuoyancyMass :: RestDraft`	Where the body's mass comes from: Rest Draft sets it at Begin Play to what the hull displaces at Rest Draft; Body Mass keeps its own (Physics → Mass), and it floats wherever that puts it, or sinks.
`RestDraftCm` cm	`15.f`	How deep the hull's lowest point sits in calm water, at rest.
`bSettleOnStart`	`true`	Held still at Begin Play until its water has been read, then set level at its rest draft on that water and released: no drop or slam at the start.
`RestAttitude`	`FRotator :: ZeroRotator`	How the body floats relative to its own frame: zero for a mesh modelled the way up it floats (a boat); a log or barrel modelled standing up floats lying down with Pitch 90. Rest Draft, the stability check and Settle On Start use it.
`MaxDepenetrationSpeed` cm/s	`100.f`	How fast physics may push the body out of something it starts in or is pressed into (rocks under the water): low, so a bad placement eases out instead of launching it. 0 leaves the project setting.
`CenterOfMassOffset` cm	`FVector :: ZeroVector`	Offset in the mesh body frame, applied as COMNudge. Scaling conventions differ from explicit pontoons; use unit-scale meshes and verify Analyze Hull.
`InertiaScale`	`FVector :: OneVector`	Scales the body's inertia (a hollow hull turns more slowly than a solid one of the same shape).
`BuoyancyMultiplier`	`1.f`	Art control on the lift. 1 is Archimedes.

Body component / Hydrodynamics [S1]

Exact property	Header default	Meaning
`PressureDragLinear`	`0.3f`	Pressure drag where a face pushes into the water: the part that grows with its speed through the water (speed / 1 m/s; Kerner's Cpd1).
`PressureDragQuadratic`	`1.f`	Pressure drag where a face pushes into the water: the part that grows with the square of its speed through the water ((speed / 1 m/s)^2; Kerner's Cpd2).
`PressureFalloff`	`0.5f`	How the pressure drag falls off as a face turns from meeting the flow head-on: it is scaled by cos^Falloff of the angle between the face's normal and the flow (Kerner's fp). Lower keeps more drag on slanted faces.

Exact property	Header default	Meaning
`SuctionDragLinear`	`0.3f`	Suction where a face pulls away from the water (behind the hull): the part that grows with its speed through the water (speed / 1 m/s; Kerner's Csd1).
`SuctionDragQuadratic`	`0.5f`	Suction where a face pulls away from the water: the part that grows with the square of its speed through the water ((speed / 1 m/s)^2; Kerner's Csd2).
`SuctionFalloff`	`0.5f`	How the suction falls off as a face turns from facing straight away from the flow: it is scaled by cos^Falloff of that angle (Kerner's fs). Lower keeps more suction on slanted faces.
`ViscousMultiplier`	`1.f`	Skin friction (ITTC 1957 line).
`SlamMultiplier`	`0.5f`	Slamming when faces drive into the water (bow slap, landing from a drop).
`FlowCoupling`	`1.f`	How much the current carries the body (0 ignores it, 1 is physical).
`AirDragCoefficient`	`0.8f`	Drag on the dry part from the air and the wind.
`AngularDamping`	`FVector :: ZeroVector`	Extra damping about the body's roll, pitch and yaw axes while wet (1/s). Zero leaves it to the water.

Body component / Propulsion [S1]

Exact property	Header default	Meaning
`bPropulsion`	`false`	A propeller and rudder (body frame). Throttle and Rudder are set from Blueprint, or the console for testing (fnb.Boat.Throttle, fnb.Boat.Rudder) when Console Control is on.
`bConsoleControl`	`true`	For testing: a change of fnb.Boat.Throttle or fnb.Boat.Rudder sets this body's Throttle and Rudder. Off: only Blueprint sets them.
`ThrustLocation` cm	`FVector :: ZeroVector`	Where the propeller pushes, in the body's frame. It pushes only while it is under the water.
`ThrustDirection`	`FVector :: ForwardVector`	Which way the propeller pushes at full ahead, in the body's frame (zero: forward, +X); astern pushes the other way.
`MaxThrustNewtons`	`1500.f`	The propeller's push at full throttle, newtons.
`PropWashSpeed` cm/s	`300.f`	The propeller's wash over the rudder at full throttle, so the boat steers when slow.
`RudderLocation` cm	`FVector :: ZeroVector`	Where the rudder sits, in the body's frame. It steers only while it is under the water.
`RudderAreaCm2`	`900.f`	The rudder blade's area, cm^2: its side force grows with it and with the square of the water's speed past the rudder (the boat's speed through the water plus the propeller's wash). 0: no rudder.
`MaxRudderAngle` deg	`35.f`	The rudder's angle at Rudder -1 or 1.
`Throttle`	`0.f`	-1 (full astern) to 1 (full ahead).

Exact property	Header default	Meaning
`Rudder`	`0.f`	-1 (hard left) to 1 (hard right).

Body component / Wake [S1]

Exact property	Header default	Meaning
`bWake`	`true`	Enable data channel wake points when the global wake setting and Interaction mode also allow them. This does not disable the separate Waves system or tracking by itself.
`Interaction`	`EFluidNinjaBuoyancyInteraction :: FluidNinjaTracking`	How the body stirs the simulation: FluidNinja's own tracking relative to the water (the default, which needs the modified FluidNinja: Project Settings → FluidNinja Buoyancy → FluidNinja Tracking; console fnb.Buoyancy.Tracking), or the plugin's wakes.
`WakeBrushSizeCm` cm	`60.f`	Brush size of each wake point at a push of 2 m/s (scaled 0.4–2x with the push).
`WakeMinPushCmS` cm/s	`20.f`	Pieces of water line pushing into the water slower than this leave no wake.
`WakeGain`	`0.15f`	Gain for data channel wake points. This is an artistic feedback control; source comments about never overshooting are not a conservation guarantee. See Feedback and effects.
`WakeStrength`	`0.35f`	How much of the hull's speed through the water (relative to the current around it) the water at the hull takes on.
`PlungeGain`	`1.f`	Gain for the outward entry burst. A symmetric authored burst can still bias the nonlinear FluidNinja river field; measure it before increasing.
`PlungeResponse`	`0.5f`	Each frame a burst point gives the water this share of what it lacks of the burst's outward speed (as Wake Gain does for the side wake): higher makes the ring start faster and harder.

Body component / Waves [S1]

Exact property	Header default	Meaning
`bWaves`	`true`	Participation in the separate wave system. Component: generate and feel waves; project or simulation: enable the corresponding wave field.
`WaveFeelScale`	`1.f`	Scales how much of the waves this body feels (κ, with Project Settings → FluidNinja Buoyancy → Waves → Wave Feel Scale): 1 is physical, 0 floats it on FluidNinja's own surface while it still makes waves.

Body component / Wet Line [S1]

Exact property	Header default	Meaning
`bWetLine`	`true`	Darkens and glosses the hull below the water line and where it was wet a moment ago, drying at Drying Speed.
`DryingSpeedCmS` cm/s	`1.5f`	How fast the wet band above the water line dries, from the top down (and the whole hull once it is out of the water).
`WetLineDataIndex`	`0`	The first of the four Custom Primitive Data slots the wet line writes on the floating primitive (this index … +3); move it when the hull's materials use those slots for something else.

Body component / Spray [S1]

Exact property	Header default	Meaning
`SplashSystem`	`Constructor / empty`	Spray where the hull slams into the water and at the bow when it drives hard into it. None: the project default (FluidNinja Buoyancy → Effects).
`SplashMinWatts`	`300.f`	Floor for repeated slam splashes. Entry splashes and bow spray have separate paths; zero does not disable all splash effects.
`MaxSplashStrength`	`12.f`	The largest a splash from this body gets: its size is the cube root of the slam's (or entry's, or bow spray's) power over Splash Reference Watts (eight times the power, twice the splash), up to this many times the splash system's own size (before Splash Scale) and never past the project's Splash Strength Ceiling.
`BowSprayMinPushCmS` cm/s	`150.f`	When nothing slams, spray is thrown where the water line drives into the water at least this fast (the bow), on the project's splash scale: the power of the water that piece of water line drives in (its area × the cube of its speed), so twice the speed is twice the spray. 0: no bow spray.
`SplashScale`	`1.f`	Scales the size of every splash and spray from this body.
`SplashInterval` s	`0.12f`	The least time between two slam splashes from this body, and between two bow sprays. A slam does not wait for the bow spray's interval (it cuts in), and a slam that could not splash yet keeps its hardest value for a quarter of a second, so one-frame slams on a spraying bow are not lost.

Body component / Sound [S1]

Exact property	Header default	Meaning
`SlapSound`	`Constructor / empty`	A one-shot played where the hull slams into the water (Slap Min Watts), louder the harder the slam.
`WashSound`	`Constructor / empty`	A loop played while the hull is in the water: the water running along the hull, louder and higher the faster it runs past the water line (Wash Full Speed) and the longer the water line; quiet lapping at rest.

Exact property	Header default	Meaning
`CreakSound`	`Constructor / empty`	A one-shot played when the hull is thrown about (Creak Min Angular Acceleration) or slams four times Slap Min Watts; "Pitch Shift" (semitones), "Impact", "Speed", "Length" and "Mass" are set on it as on the slap. None: the project default (Project Settings → FluidNinja Buoyancy → Effects; no sounds ship with the plugin).
`DripSound`	`Constructor / empty`	A one-shot played once when the hull leaves the water entirely: water dripping off it. Its float input "Impact" is set to how high the hull is wet above its last water line (cm), so a MetaSound can drip longer from a hull that went in deeper. None: no drip (there is no project default).
`Attenuation`	`Constructor / empty`	How the hull's sounds fade with distance. None: the project default (Project Settings → FluidNinja Buoyancy → Effects).
`SoundVolume`	`1.f`	Scales the volume of all of this body's hull sounds.
`SlapMinWatts`	`150.f`	Slams at least this strong (watts) play a slap; four times this also creaks. 0: no slaps, and no creaks from slams.
`WashFullSpeedCmS` cm/s	`250.f`	The wash is at full volume when the water runs past the water line this fast.
`CreakMinAngularAcceleration`	`2.f`	Creaks when the hull's spin changes faster than this (rad/s^2), or on a hard slam.

Body component / Events [S1]

Exact property	Header default	Meaning
`SlamEventWatts`	`200.f`	Slams weaker than this (watts) raise no event.

Project settings / Water [S5]

Exact property	Header default	Meaning
`bEnabled`	`true`	Floating bodies feel the water. Console: fnb.Buoyancy -1 this setting (the default), 0 off, 1 on, 2 on with the debug view.
`SurfaceTags`	`Constructor / empty`	Actors carrying one of these tags draw the water surface: FluidNinja's river spline pieces (SplineSource), its sea surfaces (SeaSurface), and any surface you tag FluidNinjaBuoyancySurface (a pool plane, a Niagara water sheet). The height capture shows only their surface primitives, leaving out a waterfall's face (steeper than 45 degrees).
`NinjaParameterCollection`	`Constructor / empty`	FluidNinja's material parameter collection: where a simulation that follows the player is (TraceMeshPos, TraceMeshSize in metres). Simulations that set those parameters on their surface material instead are found from the material.
`VelocityBufferParameter`	`("VelocityDensityBuffer")`	The surface material's texture parameter holding the simulation's velocity (RG) and density (B).

Project settings / Surface Mirror [S5]

Exact property	Header default	Meaning
`CaptureWidthCm` cm	`8000.f`	Width of the captured water square in cm. Runtime follows the configured simulation or centers relevant body bounds, selecting a nearby simulation cluster when bodies are too spread out.
`CaptureResolution`	`1024`	Texels across the capture (1024 over 80 m: 7.8 cm).
`NearPlaneAboveBodiesCm` cm	`200.f`	The capture looks down from this far above the highest floating body (or the water under it): water higher up, such as a river passing over a pool, is above its near plane and left out.
`FootprintMarginCm` cm	`100.f`	The water read back around each body: its bounds plus this, plus how far it moves in FootprintLeadSeconds.
`FootprintLeadSeconds` s	`0.15f`	See the source declaration and its owning settings page.
`bGPUGather`	`true`	Many bodies: one compute pass resamples every body's water (height and current) onto its own grid and one readback brings it all back, instead of two copies and two readbacks per body. Console: fnb.Buoyancy.Gather -1 this setting (the default), 0 the per-body copies, 1 the gather (A/B).
`MaxGatherGrid`	`128`	A body's grid points are one capture texel apart, two beyond its first hull level-of-detail distance and four beyond the second; at most this many across (a larger body is sampled more coarsely).
`MaxGatherBodies`	`256`	Bodies gathered in one frame at most; with more, they take turns.
`MaxExtrapolationSeconds` s	`0.08f`	The read-back water is a few frames old: its heights are carried forward by their rate of change for at most this long.

Project settings / Flow [S5]

Exact property	Header default	Meaning
`FlowScaleMultiplier`	`1.f`	The current (cm/s) is the velocity buffer × SimSpeed × cell size × simulation steps per second × this. The steps per second are the fixed tick of FluidNinja's NinjaLiveCore system (1/60 s: 60 steps a second at any frame rate); a NinjaLiveCore without a fixed tick steps at FluidNinja's SamplingFPS, at most the frame rate.
`MaxFlowSpeedCmS` cm/s	`600.f`	Currents faster than this are clamped (the buffer is clamped at FluidNinja's VelocityClamp).

Project settings / Physics [S5]

Exact property	Header default	Meaning
`WaterDensity`	`1000.f`	Water density (kg/m^3).
`AirDensity`	`1.2f`	Air density (kg/m^3) for the dry part of a hull.

Exact property	Header default	Meaning
`bStableSteps`	`true`	Keeps light bodies (balls, small props) stable whatever the physics step: the part of the water's spring (lift) and drag that a step is too long for is applied implicitly, as if the step had been cut finer.

Project settings / Wind [S5]

Exact property	Header default	Meaning
`Wind`	`FVector :: ZeroVector`	The wind on the dry part of hulls (cm/s, world).

Project settings / Wake [S5]

Exact property	Header default	Meaning
`bWakes`	`true`	Bodies push the water: each frame, the pieces of water line driving into it are written to FluidNinja's point input (its global Niagara Data Channel: Position, PositionPrevious, VelocityAndSize), so the simulation makes their bow waves, wakes and foam, which rock everything else.
`WakeDataChannel`	`Constructor / empty`	See the source declaration and its owning settings page.
`MaxWakePoints`	`256`	At most this many wake points a frame for all bodies (FluidNinja reads up to its DataChannelThroughput, 4000).

Project settings / FluidNinja Tracking [S5]

Exact property	Header default	Meaning
`TrackingTagProperty`	`("TrackActorPrimitiveComponentsWithTag")`	FluidNinja's own tracking, for bodies whose Interaction asks for it: the NinjaLive property (on the actor or its NinjaLive component) holding the component tag it tracks, FluidNinja's Track Actor Primitive Components With Tag. A body handed to the tracking gets that tag, besides Generate Overlap Events, which FluidNinja's tracking needs either way.
`ObjectTypeFilterProperty`	`("OverlapFilterInclusiveObjType")`	The NinjaLive property (on the actor, or failing that its NinjaLive component) listing the object types it tracks, FluidNinja's Overlap Filter Inclusive Obj Type.
`InteractionVolumeComponent`	`("InteractionVolume")`	The NinjaLive component whose bounds are where it tracks (FluidNinja's InteractionVolume). A body outside every NinjaLive's volume is not handed over until it enters one; a NinjaLive without this component selects nothing.
`RelativeTrackingSwitch`	`("TrackVelocityRelativeToWater")`	Name of the capability flag checked by the plugin. The custom core implements TrackVelocityRelativeToWater; stock FluidNinja does not.

Exact property	Header default	Meaning
`TrackedDensitySwitch`	`("SelectivelyKillBrushDensityKeepVelocity")`	How a NinjaLive says tracked bodies paint velocity but no density: FluidNinja's Selectively Kill Brush Density Keep Velocity, a number property of this name on the NinjaLive actor or one of its components, or an int user parameter of its simulation (User.Painter.<name> or User.<name>). 1 (ARRAY: primitives and bones), 4 (ARRAY and CHAOS) and...

Project settings / Effects [S5]

Exact property	Header default	Meaning
`SplashSystem`	`Constructor / empty`	Defaults for bodies that set none of their own. The spray is spawned where a hull slams into the water or drives hard into it, scaled by how hard; any Niagara system works.
`SplashReferenceWatts`	`70.f`	Splashes on one scale for every floating body and swimmer: a slam of this power (watts) makes a splash of strength 1 (the splash system at its own size), and the strength grows with the cube root of the power (eight times the power, twice the splash), so splash sizes compare across bodies, heights and stations.
`SplashStrengthCeiling`	`12.f`	The largest splash strength on the project's scale (a body's Max Splash Strength may cap its own lower).
`SlapSound`	`Constructor / empty`	Hull sounds (none ship with the plugin; see the user guide, "Hull sounds"). Slap: a one-shot when a hull slams into the water. Wash: a looping sound of water running along the hull. Creak: a one-shot when the hull is thrown about. Their volume follows the water; a float input named "Pitch Shift" (semitones) is set on each, which MetaSounds can use.
`WashSound`	`Constructor / empty`	See the source declaration and its owning settings page.
`CreakSound`	`Constructor / empty`	See the source declaration and its owning settings page.
`HullAttenuation`	`Constructor / empty`	See the source declaration and its owning settings page.

Project settings / Waves [S5]

Exact property	Header default	Meaning
`bWaves`	`true`	Participation in the separate wave system. Component: generate and feel waves; project or simulation: enable the corresponding wave field.
`WaveDepthCm` cm	`200.f`	The water's depth the waves run at where FluidNinja's height field is off (User.Wave.DepthCm); where it is on, the core takes water Z minus its bed. The waves slow and steepen in shallow water ($\omega^2 = (g\cdot k + \sigma/\rho\cdot k^3)\cdot\tanh(k\cdot d)$). Each simulation may override it, height field included (Wave Sims).
`WaveDamping`	`0.02f`	Damping of every wave (User.Wave.Damping, 1/s on the surface's vertical speed): 0.02 lets rings cross a pool.

Exact property	Header default	Meaning
`WaveSpongeCm` cm	`300.f`	Width of the absorbing band at an open simulation window's edge (User.Wave.SpongeCm): waves running out of the window fade there instead of coming back in. Used only where Wave Edge Open is on. The bodies' own fields use the same width.
`bWaveEdgeOpen`	`true`	The simulation window's border is open water (User.Wave.EdgeOpen 1: waves leave through the sponge), or a wall that reflects them (0, a walled pool). Each simulation may override it (Wave Sims).
`WaveWallAbsorption`	`0.4f`	Where a simulation's window border is a wall: how strongly it takes energy out of the waves reaching it (User.Wave.WallAbsorption, 1/s on the water's motion within Wave Wall Band of the wall), as a pool's gutter or skimmer does: a wall gives back most of a wave, not all of it. 0.4 returns about 80% of a 2 m wave's height. 0: the walls give…
`WaveWallBandCm` cm	`50.f`	Width of the band along a walled window border that Wave Wall Absorption acts in (User.Wave.WallBandCm).
`WaveShoreDepthCm` cm	`10.f`	Water shallower than this absorbs waves, more the shallower (a beach) (User.Wave.ShoreDepthCm).
`WaveBreakingSlope`	`1.f`	Slope threshold for breaking-wave damping. Current header default is 1; older 0.4 guidance is superseded.
`WaveHyperviscosity`	`0.5f`	Damping of the shortest waves the grid carries (User.Wave.Hyperviscosity, 1/s at the grid's finest wavelength, falling as k^4 for longer ones): keeps the cell-sized ripples a hull's sharp side makes from ringing.
`WaveCurrentCoupling`	`1.f`	How much FluidNinja's current carries the waves (User.Wave.CurrentCoupling): 1 sweeps them down the creek with the water, 0 leaves them in still water.
`WaveFeelScale`	`1.f`	κ: how much of the waves the floating bodies feel (times each body's own Wave Feel Scale). 1 is physical; 0 floats them on FluidNinja's own surface (the waves are still made and drawn).
`bRadiationDamping`	`true`	Restores resistance from the body's own waves in heave, roll, and pitch. Current implementation estimates motion frequency and selects damping bins, with a natural-frequency fallback.
`WaveAddedMassScale`	`1.f`	Scale for added water mass in the physics callback. Current source default is 1; use measured stability and period tests when changing it.
`bWaveDepthFromGround`	`false`	The bodies' own fields take the water's depth under each body from the ground below it (a trace down from the water), at most Wave Depth (or the simulation's override), as the core does with its landscape height field and Clamp Height Lower Values on: beds flattened by the clamp run at Wave Depth, only shallower ones at their own.
`bWaveDisplacement`	`true`	Source 1: each hull's static head, how deep it sits below the wave-free surface (open interiors included); its motion makes the impact rings, bow waves and the Kelvin wake.

Exact property	Header default	Meaning
`bWaveImpulse`	`true`	Source 2: the slam and drag that stop a hull push the same impulse into the water, equal and opposite (the Cauchy–Poisson impulse problem).
`WaveImpulseScale`	`0.35f`	Scales body impulses sent to the wave source. Current header default is 0.35; older prose describing 1 is superseded.
`WaveSourceSmoothCm` cm	`10.f`	The impulses are spread over a Gaussian this wide (σ), or three times the σ of the piece of hull each came from if that is wider, at most sixteen wave cells (under a quarter of a cell: bilinear between the four nearest cells). The impulse each puts in is unchanged.
`MaxWaveImpulses`	`32768`	The most impulse pieces (one per wet hull piece per physics step, all bodies) sent to the waves in a frame.
`bFoam`	`true`	The foam channel (needs a core at interface version 4): whitecaps where waves break, wake trails a moving hull's waterline sheds and impact rings where something lands, accumulated by the core per wave cell and drawn by the surface material; never fed back into the waves.
`FoamDecay`	`0.7f`	Exponential foam decay rate in 1/s. At 0.7 the e-folding time is 1.43 s, leaving about 37% then; it is not fully gone after that time.
`FoamBreaking`	`1.f`	Scale of the foam breaking waves make (User.Wave.FoamBreaking): the breaking decay times the water's speed. The whitecaps.
`FoamWake`	`1.f`	Scale of the foam a moving hull's waterline sheds (User.Wave.FoamWake): its static head's edge times its speed through the water. The wake trails and bow foam.
`FoamImpact`	`1.f`	Scale of the foam an impulse makes (User.Wave.FoamImpact): slams, Add Water Impulse, pokes and swimmer strokes. The impact rings.
`WaveHeightScaleParameter`	`("WaveHeightScale")`	The surface material's scalar that scales the drawn wave height (MF_NinjaOutput_BaseFunction_SURFACE's WaveHeightScale): set to 0 on the captured surfaces' dynamic materials while the Surface Mirror is drawn, so the mirror shows FluidNinja's own surface W, and put back right after.
`WaveNormalScaleParameter`	`("WaveNormalScale")`	The surface material's scalar that scales how far the waves bend the drawn normal (MF_NinjaOutput_BaseFunction_SURFACE's WaveNormalScale): set on each simulation's surface dynamic materials (Wave Normal Scale Factor, or the simulation's override).
`WaveNormalScaleFactor`	`1.f`	A simulation's surfaces get WaveNormalScale = this / (0.005 × its size in cm): 0.1 on a 20 m pool, 0.025 on an 80 m river, which keeps the drawn normal true to the waves' slope whatever the simulation's size.
`PressureBufferParameter`	`("PressureDivergenceBuffer")`	The surface material's texture parameter holding the simulation's pressure buffer (RT_PressureDivergence): on a wave simulation its B is the drawn wave height η + P and its A the static head P.
`WaveSims`	`Constructor / empty`	Per simulation: overrides by NinjaLive actor name (its name or label), for a walled pool or a simulation of its own depth.

Project settings / Swimmers [S5]

Exact property	Header default	Meaning
`bSwimmerWaves`	`true`	Swimmers push the water aside (FluidNinja Waves' static head, as a hull's): the player's pawn by its capsule, upright, and each registered swimmer by its head, chest and hips, so swimming, wading and jumping in make waves. A pawn whose capsule has no collision (seated in a boat, a boarding clip) makes none. Needs Waves.
`SwimmerVolumeLitres`	`76.f`	The swimmer's own volume, litres: the most water it pushes aside.
`bSwimmerContacts`	`true`	Swimmers' hands, forearms, feet and head (a diver's whole body) splash where they go into the water and press a ring into it (FluidNinja Waves), sized by how big they are and how fast they go in; feet kicking just under the surface churn it.
`bPlayerContacts`	`true`	The player's pawn makes body-part splashes and rings too. Off: only registered swimmers do (turn it off if the pawn's own swim animation already spawns stroke splashes).
`SwimmerEntryMinCmS` cm/s	`60.f`	A body part going in slower than this (straight down) makes no splash and no ring.
`SwimmerSplashScale`	`1.f`	Scales the body parts' splashes (1: the splash a slam of the same power would make, Splash Reference Watts).
`SwimmerImpulseScale`	`1.f`	Scales the body parts' impulses into the waves (1: the part's added mass times the speed it goes in at).
`SwimmerCavitySeconds` s	`0.35f`	A part going in less than this long after another part of the same swimmer went in nearby (Swimmer Cavity Cm) enters the air pocket that one opened: a third of the splash and half the impulse.
`SwimmerCavityCm` cm	`30.f`	How near (across the surface) a part must go in to another's entry to enter its air pocket (Swimmer Cavity Seconds).
`SwimmerKickDepthCm` cm	`25.f`	Feet moving up and down at most this deep under the surface churn it (a flutter kick), more the nearer they are.
`SwimmerStrokeScale`	`1.f`	Scales the strokes' push on the water (FFluidNinjaSwimmerPose::StrokeImpulse, the FluidNinja Swimmer component's Add Stroke Impulse): the wake point a stroke writes into FluidNinja's point input and the small impulse it presses into the waves. 0: strokes do nothing.
`bSwimmerWakes`	`true`	Swimmers write their push through the water into FluidNinja's point input as moving brush points (the Wake Data Channel, as the floating bodies' wakes), and a ring pushing outward where they plunge in.
`SwimmerWakeStrength`	`0.35f`	The wake's target: the current plus this share of the swimmer's speed through the water (a hull's Wake Strength). Each frame the wake writes Swimmer Wake Gain of what the water at the swimmer still lacks of it.
`SwimmerWakeGain`	`0.2f`	Share of what the water at a swimmer still lacks of the target that the wake writes each frame (a hull's Wake Gain).

Exact property	Header default	Meaning
`SwimmerWakeSizeCm` cm	`70.f`	Brush size of a swimmer's wake (it grows a little with speed). Wading uses two thirds of it at each foot, a plunge 1.2 times it.
`SwimmerWakeMinSpeedCmS` cm/s	`15.f`	A swimmer moving slower than this through the water leaves no wake.
`SwimmerCurrentRingInnerCm` cm	`70.f`	The current a swimmer is in is read on a ring around it, not under it, where its own wake is: from this far... The wakes read it there (they push only what the water around them lacks), and so does UFluidNinjaBuoyancySubsystem::GetCurrentAtLocation by default.
`SwimmerCurrentRingOuterCm` cm	`160.f`	...to this far (at least 10 cm past the inner radius).

Simulation component / FluidNinja Waves [S14]

Exact property	Header default	Meaning
`bWaves`	`true`	Participation in the separate wave system. Component: generate and feel waves; project or simulation: enable the corresponding wave field.
`DepthCm` cm	`200.f`	This simulation's water depth for the waves everywhere in it (User.Wave.DepthCm), instead of the project's Wave Depth and of the depth FluidNinja's height field would give: the waves slow and steepen in shallow water. Read only while Depth From Bed is off.
`bDepthFromBed`	`false`	The core takes the depth from FluidNinja's height field where it has one (water Z minus its bed, User.Wave.DepthFromBed), and the project's Wave Depth where it has none; Depth Cm is then not used. Off (the default): Depth Cm everywhere, for a pool or channel whose height field reads its water surface rather than its floor.
`bWalled`	`false`	A closed basin: the simulation window's border is a wall that reflects the waves (User.Wave.EdgeOpen 0) and takes energy out of them within Wall Band of it at Wall Absorption, as a pool's gutter does. Off: open water, where waves leave through the sponge.
`WallAbsorption`	`0.4f`	How strongly a walled border takes energy out of the waves reaching it (User.Wave.WallAbsorption, 1/s on the water's motion within Wall Band of the wall): 0.4 returns about 80% of a 2 m wave's height; 0 returns everything, and a closed basin then keeps every wave a floater makes.
`WallBandCm` cm	`50.f`	Width of the band along a walled border that Wall Absorption acts in (User.Wave.WallBandCm).
`ScaleDownOverride`	`0`	The wave grid's scale-down for this simulation, 1, 2 or 4 (User.Wave.ScaleDown): its waves run on its pressure buffer's cells over this, so a tick costs a quarter at 2 and a sixteenth at 4, and the waves are drawn that much coarser. 0: the console variable fnb.Waves.ScaleDown, which the quality tiers (sg.WaterQuality) set.

Exact property	Header default	Meaning
`Importance`	`1.f`	How much this simulation matters when fnb.Waves.MaxActiveSims keeps only the simulations nearest the camera running waves: its distance from the camera is divided by this before the simulations are ranked, so at 2 it counts as half as far away (the player's pool over a distant pond).

Preset asset / Preset [S17]

Exact property	Header default	Meaning
`Description`	`Constructor / empty`	A line for the menu and the details panel: what this preset makes of an object.

Preset asset / Hull [S17]

Exact property	Header default	Meaning
`Hull`	`EFluidNinjaPresetHull :: FromCollision`	How the hull is built from the object's mesh.
`HullResolutionCm` cm	`25.f`	From Collision: the hull surface's resolution (cm). Finer follows the shape better and costs more per step.
`VoxelCount`	`64`	Voxels: how many cubes fill the collision.
`SphereCount`	`8`	Sphere Row and Sphere Ring: how many spheres.
`SphereRadiusScale`	`1.f`	The spheres' radius, as a multiple of the radius fitted to the object (One Sphere: the mean of its three half-extents; a row: the object's thinner half-extent; a ring: half the tube's thickness).

Preset asset / Mass [S17]

Exact property	Header default	Meaning
`Mass`	`EFluidNinjaPresetMass :: Density`	See the source declaration and its owning settings page.
`DensityKgM3`	`500.f`	Density: what the object is made of, kg/m³ (water 1000; pine 500, cork 240, ice 917, iron 7800; a beach ball about 8). Above 1000 it sinks.
`RestDraftCm` cm	`10.f`	Rest Draft: how deep it floats at rest (cm).

Preset asset / Balance [S17]

Exact property	Header default	Meaning
`Attitude`	`EFluidNinjaPresetAttitude :: AsPlaced`	Which way up it rests.

Exact property	Header default	Meaning
`CentreOfMassShift`	`0.f`	Moves the centre of mass along the object's longest axis, as a share of its half-length (+0.3: a third of the way toward the axis' positive end). An arrow's steel head makes it float nose down; a bottle's base makes it stand up. 0 leaves it centred.
`bSettleOnStart`	`true`	Put on the water at its rest draft and attitude when play starts, rather than dropped where it was placed.
`bSleepWhenSettled`	`true`	Sleep once it has come to rest on still water, until something moves it or the water does.

Preset asset / Hydrodynamics [S17]

Exact property	Header default	Meaning
`PressureDragLinear`	`0.3f`	See the source declaration and its owning settings page.
`PressureDragQuadratic`	`1.f`	See the source declaration and its owning settings page.
`SuctionDragLinear`	`0.3f`	See the source declaration and its owning settings page.
`SuctionDragQuadratic`	`0.5f`	See the source declaration and its owning settings page.
`AirDragCoefficient`	`0.8f`	See the source declaration and its owning settings page.

Preset asset / Water [S17]

Exact property	Header default	Meaning
`bSetInteraction`	`false`	Also set how the body stirs FluidNinja (Interaction). Off, the component keeps its project's default: a preset made for one project should not hand bodies to FluidNinja's tracking in a project whose NinjaLives do not track.
`Interaction`	`EFluidNinjaBuoyancyInteraction :: Wakes`	The plugin's wakes, FluidNinja's own tracking (the modified core), or both.
`bWake`	`true`	Enable data channel wake points when the global wake setting and Interaction mode also allow them. This does not disable the separate Waves system or tracking by itself.
`bWetLine`	`true`	Show the wet line on the hull (the hull material needs MF_FNB_HullWetness).

Preset asset / Spray [S17]

Exact property	Header default	Meaning
`SplashMinWatts`	`20.f`	Floor for repeated slam splashes. Entry splashes and bow spray have separate paths; zero does not disable all splash effects.
`MaxSplashStrength`	`12.f`	The largest splash this body throws, on the project's splash scale.

Exact property	Header default	Meaning
`SplashScale`	`1.f`	An art multiplier on every splash of this body.

Pontoon fields

Field	Default	Meaning
Location	(0,0,0)	Already-scaled body-frame cm.
Radius	25 cm	Already-scaled sphere radius.
LiftWeight	1	Scales that sphere's buoyant contribution.
DragWeight	1	Scales that sphere's drag contribution.

21 · EVIDENCE AND TERMINOLOGY

Source map and glossary

The source map lists paths relative to the FluidNinja Buoyancy plugin package. Locate the same paths in your installed plugin or source distribution; no particular project folder or installation path is assumed. `source_manifest.json` records SHA-256 values of the inspected text sources so a maintainer can identify differences. The guide itself is self-contained, and no Unreal binary package was read to create it.

- **S1 · Shared buoyancy component**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancyComponent.h`
- **S2 · Hull construction and hydrodynamic integration**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyCore.cpp`
- **S3 · Physics callback and stable steps**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyCallback.cpp`
- **S4 · Surface capture and runtime orchestration**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancySubsystem.cpp`
- **S5 · Global settings and wave controls**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancySettings.h`
- **S6 · Wave coupling and self fields**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyWaves.cpp`
- **S7 · Wave step shader**
  `Tools/CorePatch/FluidNinjaWaves/Ninja_WaveStep.hlsl`
- **S8 · Wave export shader**
  `Tools/CorePatch/FluidNinjaWaves/Ninja_WaveExport.hlsl`
- **S9 · Core patch tooling**
  `Tools/CorePatch/apply_fluidninja_waves.py`
- **S10 · Relative tracking design**
  `Docs/RelativeTrackingProposal.md`
- **S11 · Preset definitions**
  `Tools/make_presets.py`
- **S12 · Preset application**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyPresetLibrary.cpp`
- **S13 · Unified water query**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancyWaterQuery.h`
- **S14 · Per simulation wave settings**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaWaveSimComponent.h`
- **S15 · Source gather shader**
  `Shaders/Private/FluidNinjaBuoyancyGather.usf`
- **S16 · Wave source and self field shader**
  `Shaders/Private/FluidNinjaBuoyancyWaveSource.usf`
- **S17 · Preset data asset schema**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancyPreset.h`
- **S18 · Component setup and effects**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyComponent.cpp`
- **S19 · Quality tier configuration**
  `Config/DefaultFluidNinjaBuoyancy.ini`
- **S20 · Developer test bench**
  `Source/FluidNinjaBuoyancyDeveloper/Private/FluidNinjaBuoyancyTestBench.cpp`

- **S21 · Public subsystem and integration API**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancySubsystem.h`
- **S22 · Swimmer component API**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancySwimmerComponent.h`
- **S23 · Module registration and settings constructor**
  `Source/FluidNinjaBuoyancy/Private/FluidNinjaBuoyancyModule.cpp`
- **S24 · Swimmer movement adapter interface**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancySwimmerProvider.h`
- **S25 · Swimmer skeleton and contact profile**
  `Source/FluidNinjaBuoyancy/Public/FluidNinjaBuoyancySwimmerProfile.h`

Primary background references

- FluidNinja LIVE 2 manual, section 14.8 and model limitations. Consulted 18 October 2026; the guide's custom capabilities are established by local code, not attributed to the vendor.
- NASA Glenn on Archimedes' principle: buoyancy equals displaced fluid weight.
- NASA Glenn on the drag equation: density, speed squared, reference area, and coefficient.
- Epic on physics sub-stepping: smaller time steps trade CPU cost for better-resolved physics. The plugin's Stable Steps implementation is separately documented by source.

Glossary

Term	Plain meaning
Draft / freeboard	Depth of the hull's lowest point below water / height of the deck or rim above water.
Displacement	Water volume excluded by the immersed hull; also used for the corresponding supported mass.
Heave / roll / pitch / yaw	Vertical translation / side-to-side tilt / bow-up-and-down tilt / horizontal turning.
Hydrostatics / hydrodynamics	Support and balance at rest / forces from relative motion.
Readback / patch	Asynchronously copied GPU data / a body-specific grid of sampled water.
Static head	The equivalent water-depth source produced by the immersed hull footprint.
Radiation damping	Loss of body motion to waves it generates.
Added mass	Extra inertia from water moving with the body, without extra gravitational weight.
WPO	World Position Offset: material displacement of rendered vertices.
Own field	Local simulation of one body's wave contribution, removed from its delayed feedback.